



---

# Rapport de Projet : MLP et moteur d'autodiff appliqués à MNIST

---

*Auteurs:*

Jianye Shi, Hao Qian, Xiang Bian, Abdenmour Boulmis

*Encadrant:*

Aurélien Delval

*Responsables:*

Aurélien Delval / Hugo Bolloré / Pablo Oliveira

December 19, 2025

# Sommaire

|          |                                                                          |           |
|----------|--------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                      | <b>4</b>  |
| <b>2</b> | <b>Contexte et État de l’art</b>                                         | <b>4</b>  |
| 2.1      | Introduction au problème . . . . .                                       | 4         |
| 2.2      | Concept de la méthode classique . . . . .                                | 4         |
| 2.3      | Présentation des éléments modulables du MLP . . . . .                    | 6         |
| 2.3.1    | Fonctions d’activation . . . . .                                         | 6         |
| 2.3.2    | Algorithmes de descente de gradient . . . . .                            | 6         |
| 2.4      | Positionnement et Périmètre . . . . .                                    | 7         |
| <b>3</b> | <b>Implémentation</b>                                                    | <b>8</b>  |
| 3.1      | Architecture Globale . . . . .                                           | 8         |
| 3.2      | Noyau de Calcul : Matrix . . . . .                                       | 9         |
| 3.3      | Autodifférentiation : Node et MathOps . . . . .                          | 9         |
| 3.4      | Modélisation et Entraînement . . . . .                                   | 9         |
| <b>4</b> | <b>Expérimentations</b>                                                  | <b>11</b> |
| 4.1      | Configuration de l’environnement . . . . .                               | 11        |
| 4.1.1    | Environnement Matériel / Hardware . . . . .                              | 11        |
| 4.1.2    | Environnement Logiciel / Software . . . . .                              | 11        |
| 4.1.3    | Configuration Spécifique et Protocole . . . . .                          | 11        |
| 4.2      | Analyse des performances de calcul et d’entraînement . . . . .           | 12        |
| 4.2.1    | Passage à l’échelle (Thread Scaling) . . . . .                           | 12        |
| 4.2.2    | Hiérarchie des performances et Speedup . . . . .                         | 14        |
| 4.2.3    | Analyse de Profiling et Loi d’Amdahl : vers l’entraînement complet . . . | 15        |
| 4.3      | Convergence de l’entraînement . . . . .                                  | 16        |
| 4.3.1    | Premiers tests de convergence . . . . .                                  | 16        |
| 4.3.2    | Impact du Taux d’Apprentissage (Learning Rate) . . . . .                 | 17        |
| 4.3.3    | Impact de la Taille de Batch (Batch Size) . . . . .                      | 18        |
| 4.3.4    | Impact de la Taille de la Couche Cachée (Hidden Size) . . . . .          | 19        |
| 4.3.5    | Impact de la Fonction d’Activation et de l’Initialisation . . . . .      | 20        |
| 4.3.6    | Impact de l’Affinité sur les charges d’entrée (Input Layer) . . . . .    | 22        |
| 4.4      | Synthèse des Résultats Expérimentaux . . . . .                           | 22        |
| <b>5</b> | <b>Déroulé du Projet</b>                                                 | <b>23</b> |
| <b>6</b> | <b>Conclusion &amp; perspectives</b>                                     | <b>23</b> |
| 6.1      | Conclusion : . . . . .                                                   | 23        |
| 6.2      | Perspectives : . . . . .                                                 | 23        |
| <b>7</b> | <b>Références</b>                                                        | <b>25</b> |
| <b>8</b> | <b>Glossaire</b>                                                         | <b>25</b> |
| <b>A</b> | <b>Exemple : graphe de calcul et autodiff</b>                            | <b>27</b> |
| <b>B</b> | <b>Protocole de Verrouillage des Fréquences</b>                          | <b>27</b> |
| <b>C</b> | <b>Scripts d’Expérimentation</b>                                         | <b>27</b> |

## List of Figures

|    |                                                                                                                                                                                                                                                        |    |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1  | Architecture d'un Perceptron Multicouche (MLP) avec couches cachées [3]. . . . .                                                                                                                                                                       | 5  |
| 2  | Architecture logicielle : du calcul matriciel (bas niveau) au pipeline d'entraînement. . . . .                                                                                                                                                         | 8  |
| 3  | Temps d'exécution en fonction du nombre de threads pour différentes tailles de matrices. . . . .                                                                                                                                                       | 12 |
| 4  | Speedup (accélération) en fonction du nombre de threads. . . . .                                                                                                                                                                                       | 13 |
| 5  | Comparaison des temps d'exécution (Naïf vs Reordered vs OpenMP vs BLAS). . . . .                                                                                                                                                                       | 14 |
| 6  | Speedup relatif des différentes optimisations. . . . .                                                                                                                                                                                                 | 14 |
| 7  | Répartition du temps d'exécution (End-to-End) : Naïf vs BLAS. . . . .                                                                                                                                                                                  | 16 |
| 8  | Courbe de <i>loss</i> et de précision ( <i>accuracy</i> ) sur les données d'entraînement ( <b>60 000 images</b> ) et de validation ( <b>10 000 images</b> ). . . . .                                                                                   | 17 |
| 9  | Impact du <i>learning rate</i> sur la convergence (10 époques, 3 graines). Les axes de précision utilisent une échelle discontinue pour visualiser simultanément les hautes précisions ( $> 90\%$ ) et les échecs de convergence ( $< 40\%$ ). . . . . | 18 |
| 10 | Impact de la Taille de Batch (20 époques, $LR = 0.01$ , 3 graines). Les petits batches (16, 32) convergent mieux mais sont plus lents à exécuter. Le batch 256 diverge avec ce $LR$ fixe. . . . .                                                      | 19 |
| 11 | Impact de la Taille de la Couche Cachée (20 époques, $LR = 0.01$ , $B = 64$ , 3 graines). L'augmentation de la taille améliore la précision, mais avec des rendements décroissants au-delà de 128 neurones. . . . .                                    | 20 |
| 12 | Comparaison des Fonctions d'Activation et Initialisation. ReLU (He) et ReLU (Manual) offrent les meilleures performances. Sigmoid et Tanh (avec Xavier) restent compétitifs grâce à une initialisation adaptée. . . . .                                | 21 |

## List of Tables

|   |                                                                                                                    |    |
|---|--------------------------------------------------------------------------------------------------------------------|----|
| 1 | Configuration matérielle . . . . .                                                                                 | 11 |
| 2 | Configuration logicielle . . . . .                                                                                 | 11 |
| 3 | Profil d'exécution (gprof) - Version Naïve : <code>Matrix::matmul</code> domine (96.4%) . . . . .                  | 15 |
| 4 | Profil d'exécution (gprof) - Version BLAS : <code>matmul</code> négligeable, overhead mémoire dominant . . . . .   | 16 |
| 5 | Configuration standard pour le test de convergence. . . . .                                                        | 17 |
| 6 | Impact de l'affinité sur les performances pour une multiplication matricielle $64 \times 784 \times 128$ . . . . . | 22 |

## 1 Introduction

Ce rapport présente l'implémentation *from scratch* d'un Perceptron Multicouche (MLP) et d'un moteur de différenciation automatique appliqué au jeu de données MNIST [2]. Réalisé dans le cadre du Master 1 CHPS à l'Université Paris-Saclay, ce projet vise un double objectif : comprendre les mécanismes internes de l'apprentissage profond (propagation, rétropropagation) et s'initier à l'optimisation de code bas niveau, sans dépendre de bibliothèques tierces.

## Plan du document

Le rapport s'articule autour de quatre axes :

1. **État de l'art** : présentation du problème MNIST et des fondements théoriques du MLP.
2. **Implémentation** : architecture du moteur d'autodifférentiation et conception logicielle.
3. **Expérimentations** : validation de la convergence (Machine Learning) et analyse des performances (HPC).
4. **Organisation** : répartition des tâches et déroulement du projet.

Une conclusion, des annexes techniques et un glossaire complètent ce document.

## 2 Contexte et État de l'art

### 2.1 Introduction au problème

Au cours des dernières décennies, l'apprentissage automatique a connu un développement considérable, principalement grâce aux progrès réalisés dans le domaine des réseaux de neurones artificiels. Parmi les problèmes emblématiques de ce domaine figure la reconnaissance de chiffres manuscrits, couramment étudiée à l'aide du jeu de données MNIST [2], devenu un standard pour les tâches de classification. Ce travail vise à présenter les approches classiques permettant de résoudre ce problème, en commençant par le perceptron multicouche (MLP), avant d'aborder ses principales variantes et l'évolution des modèles plus récents.

### 2.2 Concept de la méthode classique

Le problème MNIST consiste à classer des images de chiffres manuscrits ( $28 \times 28$  pixels) dans dix catégories correspondant aux chiffres 0 à 9. Pour résoudre ce problème, on utilise un perceptron multicouche (MLP), c'est-à-dire un réseau de neurones constitué de plusieurs neurones artificiels organisés en couches (entrée, cachées, sortie).

Chaque neurone calcule une combinaison linéaire de ses entrées, à laquelle il applique ensuite une fonction d'activation non linéaire. Soit  $x$  le vecteur d'entrée,  $w$  le vecteur de poids et  $b$  le biais. La sortie  $y$  d'un neurone est donnée par :

$$y = \sigma(w \cdot x + b)$$

**Passage au calcul matriciel** On commence par le cas d'un seul exemple. Une image MNIST de taille  $28 \times 28$  est vectorisée (*flattened*) en un vecteur d'entrée  $x \in \mathbb{R}^{1 \times 784}$ . Pour une couche

entièrement connectée comportant  $n$  neurones, on introduit une matrice de poids  $W \in \mathbb{R}^{784 \times n}$  et un biais  $b \in \mathbb{R}^{1 \times n}$ . La pré-activation s'écrit alors :

$$z = xW + b$$

où  $z \in \mathbb{R}^{1 \times n}$ . Cette transformation correspond à une application affine de l'espace des pixels (dimension 784) vers un espace de représentation de dimension  $n$ , avant l'application de la non-linéarité  $\sigma$ .

Dans la pratique, un traitement séquentiel (image par image) exploite imparfaitement les ressources matérielles. Afin d'augmenter le débit (*throughput*), on recourt au calcul par *mini-batches* (*mini-batches*), qui permet de reformuler une suite de produits vecteur-matrice en un unique produit matrice-matrice, particulièrement optimisé (SIMD, multi-cœurs, BLAS, et, le cas échéant, GPU).

Plus précisément, en regroupant  $N$  exemples, on définit la matrice d'entrées  $X \in \mathbb{R}^{N \times 784}$ . La propagation avant se généralise en :

$$Y = \sigma(XW + \mathbf{1}_N b)$$

où  $\mathbf{1}_N \in \mathbb{R}^{N \times 1}$  désigne un vecteur de 1 et  $\mathbf{1}_N b$  formalise la diffusion (*broadcast*) du biais sur l'ensemble des  $N$  exemples.

Pour clarifier ce mécanisme, la figure suivante illustre d'abord le cas  $xW$ , puis son extension naturelle au cas matriciel avec  $XW$ .

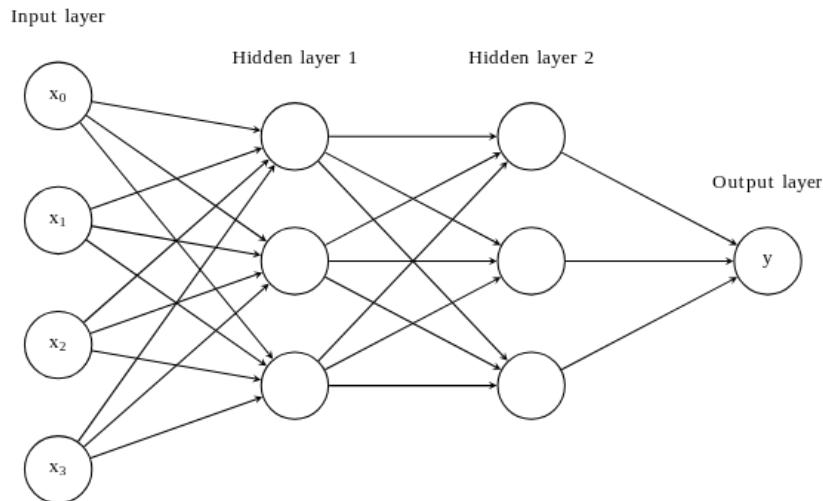


Figure 1: Architecture d'un Perceptron Multicouche (MLP) avec couches cachées [3].

Cette figure illustre la structure hiérarchique du modèle (schéma purement illustratif : pour MNIST, chaque entrée est un vecteur de dimension 784, et un mini-batch correspond à une matrice  $X$  de dimensions  $N \times 784$ ) :

**Couche d'entrée** ( $x_0$  à  $x_3$ ) : reçoit les données brutes (ici les pixels d'une image).

**Couches cachées** : couches intermédiaires où le réseau apprend des représentations abstraites ; chaque neurone est connecté à tous les neurones de la couche précédente (*fully connected*).

**Couche de sortie** ( $y$ ) : produit la prédiction finale (probabilités des classes pour MNIST).

Bien que le schéma montre des connexions individuelles, notre implémentation vectorise ces opérations sous forme de produits matriciels  $X \cdot W$ . C'est cette opération matricielle que nous optimisons intensivement dans ce projet (BLAS, OpenMP).

L'objectif de l'apprentissage est de minimiser une fonction de *loss* (*loss*)  $L$ , qui quantifie l'écart entre les prédictions du modèle et les étiquettes réelles. Minimiser  $L$  revient donc à

réduire cet écart sur les données d'entraînement (et, idéalement, à améliorer la capacité de généralisation sur des exemples non vus). Dans la suite, la rétropropagation (*backpropagation*) [4] permet de calculer le gradient  $\nabla L$  par rapport à tous les paramètres afin d'effectuer cette minimisation.

**Règle de la chaîne (*chain rule*)** Le réseau peut se voir comme une composition de fonctions  $f(x) = f_n(f_{n-1}(\dots f_1(x)))$ . Le gradient se calcule en remontant de la fin vers le début via la règle de la chaîne :

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial x}$$

Cela conduit naturellement à une représentation sous forme de **graphe de calcul** (DAG), où chaque nœud est une opération élémentaire, connaissant ses parents et sa dérivée locale. Lors de la phase *backward*, nous propageons le gradient accumulé depuis la perte jusqu'aux feuilles (les poids). Dans le cas général, exprimer analytiquement le gradient d'un réseau profond est difficile et source d'erreurs. L'autodiff permet de le calculer automatiquement en appliquant systématiquement la règle de la chaîne, sans dériver explicitement les formules pour chaque architecture (p. ex. PyTorch, TensorFlow).

**Exemple : graphe de calcul et autodiff** Un exemple détaillé est donné en Annexe A.

## 2.3 Présentation des éléments modulables du MLP

Après avoir décrit le fonctionnement général du MLP et les principes du calcul de gradient, nous présentons maintenant les différents éléments modulables de cette architecture.

### 2.3.1 Fonctions d'activation

Les fonctions d'activation introduisent la non-linéarité dans le réseau. Elles sont cruciales pour deux raisons :

1. **Non-linéarité** : Sans elles, une superposition de couches linéaires resterait une simple transformation linéaire unique ( $W_2(W_1x) = W'x$ ), incapable de modéliser des problèmes complexes (comme XOR ou MNIST).
2. **Recadrage des valeurs** : Elles permettent souvent de maintenir les activations dans une plage bornée (p. ex.  $[0,1]$  pour la sigmoïde), évitant l'explosion des valeurs lors de la propagation dans des réseaux profonds.

**Sigmoïde** : transforme l'entrée en une valeur entre 0 et 1, mais peut provoquer la saturation des gradients.

**Tanh** : centrée sur zéro, elle améliore souvent la convergence mais reste sujette au même problème de saturation.

**ReLU** (*Rectified Linear Unit*) : ne conserve que les valeurs positives ( $ReLU(x) = \max(0, x)$ ) ; elle accélère l'apprentissage et constitue aujourd'hui un choix standard [1].

### 2.3.2 Algorithmes de descente de gradient

L'apprentissage consiste à minimiser la *loss* grâce à la descente de gradient. L'algorithme met à jour les paramètres  $\theta$  (poids et biais) itérativement :

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_{\theta} L$$

où  $\eta$  est le **taux d'apprentissage** (*learning rate*).

- Un  $\eta$  trop grand accélère la convergence mais risque de provoquer des oscillations, voire une divergence.
- Un  $\eta$  trop petit garantit la stabilité mais ralentit considérablement l'apprentissage.

Listing 1: Algorithme simplifié (SGD)

```
Pour chaque epoch :  
  Pour chaque mini-batch (X, Y) :  
    1. Forward: P = Model(X)  
    2. Loss: L = LossFn(P, Y)  
    3. Backward: Calculer les gradients via autodiff  
    4. Update: W = W - learning_rate * dL/dW
```

Si ce projet se concentre sur la méthode **SGD**, il convient de citer les évolutions majeures de l'état de l'art :

- **SGD** (*Stochastic Gradient Descent*) : met à jour les poids après chaque *mini-batch* (**méthode implémentée**).
- **Momentum** : ajoute une inertie pour lisser les oscillations et accélérer la convergence.
- **Adam** : combine Momentum et adaptation du taux d'apprentissage ; optimisateurs standards actuels.

## 2.4 Positionnement et Périmètre

Bien que les réseaux convolutionnels (CNN), introduits par LeCun *et al.* (LeNet-5 [2]), dominent la reconnaissance d'images grâce à leur invariance spatiale (> 99% de réussite), le Perceptron Multicouche (MLP) reste le modèle pédagogique de référence. Il permet de maîtriser les concepts fondamentaux : calcul matriciel, non-linéarité et rétropropagation du gradient.

Notre projet se concentre ainsi sur l'implémentation *from scratch* d'un pipeline complet :

- a) moteur de tenseurs et d'autodifférentiation,
- b) architecture modulaire (couches, activations, optimiseur, loss),
- c) boucle d'entraînement optimisée (SGD, CrossEntropy, DataLoader, MiniBatch).

### 3 Implémentation

#### 3.1 Architecture Globale

Le projet implémente un moteur de *Deep Learning* complet en C++ moderne, privilégiant la compréhension des mécanismes bas niveau et la performance. Le choix du C++ est motivé par son contrôle fin de la mémoire et sa capacité à produire un code efficace, en cohérence avec l'orientation *calcul haute performance* de l'UE. L'architecture (Fig. 2) s'articule autour de trois couches : le noyau de calcul (**Matrix**), le graphe d'autodiff (**Node/MathOps**) et les abstractions de plus haut niveau (couches, *loss*, optimiseur, boucle d'entraînement).

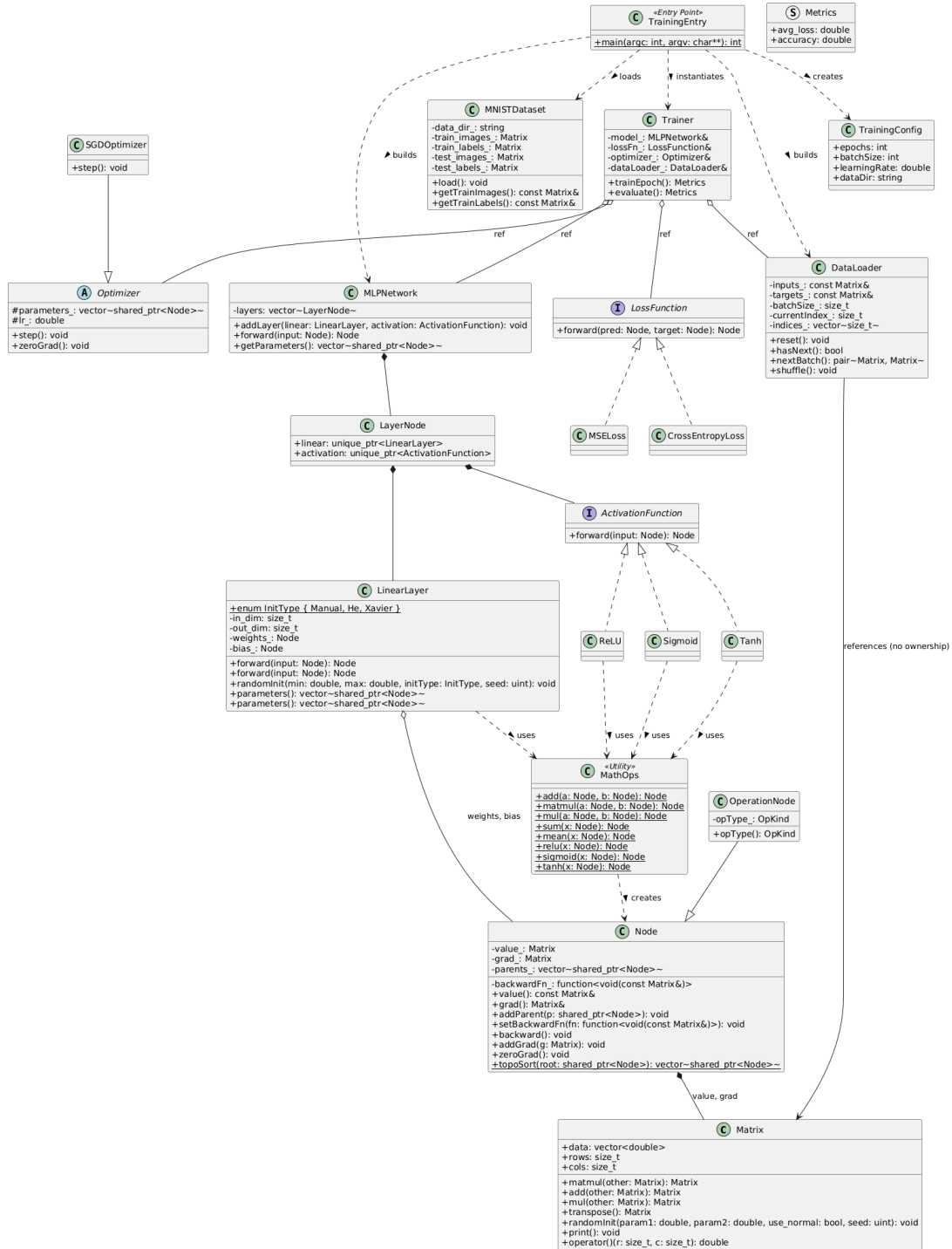


Figure 2: Architecture logicielle : du calcul matriciel (bas niveau) au pipeline d'entraînement.



### 3.2 Noyau de Calcul : Matrix

La classe `Matrix` constitue la brique fondamentale. Pour optimiser l'utilisation des caches CPU (localité spatiale) et permettre la vectorisation (SIMD), les données sont stockées dans un conteneur contigu (`std::vector<double>`). Afin d'analyser l'impact des optimisations, nous avons implémenté plusieurs versions de la multiplication matricielle (`matmul`), sélectionnables à l'exécution : une version naïve (triple boucle) à but pédagogique, et des versions performantes exploitant OpenMP et les bibliothèques BLAS. L'initialisation des poids suit des stratégies robustes (He pour ReLU, Xavier pour Sigmoid/Tanh) et intègre une gestion déterministe de l'aléatoire (*seed*) pour assurer la reproductibilité.

Concrètement, la méthode `randomInit` applique une mise à l'échelle dépendante du fan-in : pour He,  $\text{Var}(W) = 2/n_{in}$  (adaptée à ReLU) ; pour Xavier,  $\text{Var}(W) = 1/n_{in}$  (adaptée à Sigmoid/Tanh). La graine est explicitement contrôlée afin de reproduire à l'identique les poids initiaux entre deux exécutions.

### 3.3 Autodifférentiation : Node et MathOps

Le moteur d'*autodiff* repose sur un Graphe Acyclique Dirigé (DAG) dynamique. Chaque `Node`, géré via des pointeurs intelligents (`std::shared_ptr`), stocke sa valeur, son gradient et ses dépendances. L'originalité de l'implémentation réside dans l'utilisation d'expressions Lambda pour définir la rétropropagation (*backpropagation*). Lors du passage avant (*forward*), le contexte local est capturé efficacement par la fermeture (*closure*), évitant la création de classes dédiées pour chaque opération.

Listing 2: Rétropropagation via Lambda (MathOps)

```
// Exemple pour Mul (y = a * b) : capture du contexte {a, b}
node->setBackwardFn([a, b](const Matrix& grad){
    // Règle de la chaîne : dL/da = grad * b, dL/db = grad * a
    a->addGrad(grad.mul(b->value()));
    b->addGrad(grad.mul(a->value()));
});
```

La rétropropagation est déclenchée par un tri topologique du graphe (méthode `topoSort`), garantissant un parcours correct des dépendances sans récursion profonde. Pour faciliter l'analyse du graphe, les nœuds d'opérations sont identifiés par un type (`OperationNode / OpKind`). Enfin, certaines opérations (p. ex. `add`) gèrent une diffusion (*broadcasting*) simple afin de supporter l'ajout du biais sur un mini-batch.

### 3.4 Modélisation et Entraînement

Le réseau est construit par composition de modules. **Couches** : `LinearLayer` encapsule les paramètres (poids, biais), et l'interface `ActivationFunction` permet d'interchanger les non-linéarités (ReLU, Sigmoid, Tanh) via le polymorphisme. La combinaison « couche linéaire + activation » est encapsulée dans `LayerNode`, et le modèle complet est représenté par `MLPNetwork` (séquence de `LayerNode`).

**Entraînement** : `Trainer` orchestre la boucle d'apprentissage. Les données sont chargées par `MNISTDataset` et regroupées en mini-batches via `DataLoader`. La *loss* est définie par l'interface `LossFunction` (implémentations `MSELoss` et `CrossEntropyLoss`), tandis que la mise à jour des paramètres est assurée par `Optimizer` (implémentation `SGDOptimizer`). À chaque mini-batch, le pipeline exécute : *forward* → calcul de la *loss* → *backward* → *step*().

Du point de vue du gradient, `MSELoss` conduit à un terme proportionnel à  $(\hat{y} - y)$ , tandis que `CrossEntropyLoss` est implémentée via un *softmax* numériquement stable ; le gradient par rapport aux *logits* s'écrit alors simplement  $\nabla_z L = p - y$  (probabilités prédites moins labels *one-hot*), ce qui favorise une rétropropagation efficace.

Cette conception modulaire permet d'étendre le framework avec de nouvelles fonctions de *loss* ou d'optimisation, sans modifier le moteur d'autodiff sous-jacent.

**Flux de données (mini-batch)** Lors de l'entraînement, `DataLoader` fournit un mini-batch sous forme de matrices  $(X, Y)$ . Le modèle `MLPNetwork` effectue la propagation avant pour produire les *logits*, puis `LossFunction` construit un nœud scalaire représentant la *loss* du mini-batch. L'appel `backward` déclenche un tri topologique (`topoSort`) et propage les gradients depuis la *loss* jusqu'aux paramètres. Enfin, `Optimizer::step()` met à jour les poids et biais à partir des gradients accumulés.

**Choix de conception** Nous avons retenu un graphe de calcul dynamique (DAG) afin de représenter explicitement les dépendances entre opérations et de supporter des compositions arbitraires. Les nœuds sont gérés via `std::shared_ptr` pour simplifier le cycle de vie (graphes non-linéaires) et réduire le risque de fuites mémoire. Les gradients locaux sont définis via des lambdas capturant le contexte, ce qui limite le *boilerplate* et facilite l'ajout d'opérations. La rétropropagation s'appuie sur un parcours itératif (tri topologique), évitant la récursion profonde. Enfin, le noyau `matmul` dispose d'implémentations interchangeables (naïf/OpenMP/BLAS) pour concilier objectif pédagogique et performance.

**Points saillants de l'implémentation** La sélection de l'implémentation `matmul` est effectuée à l'exécution, ce qui permet des comparaisons directes entre versions. L'ajout du biais est pris en charge via un *broadcasting* simple dans `MathOps::add`. Les expériences sont rendues reproductibles grâce au contrôle explicite de la graine (*seed*) pour l'initialisation et le *shuffling*. Enfin, `CrossEntropyLoss` combine un *softmax* stable et un gradient de forme simple  $(p - y)$  pour assurer une rétropropagation robuste.

## 4 Expérimentations

### 4.1 Configuration de l’environnement

Toutes les expériences ont été réalisées sur une machine avec les spécifications suivantes :

#### 4.1.1 Environnement Matériel / Hardware

Table 1: Configuration matérielle

| Composant  | Caractéristiques                                                                                                                                                                                                                                                                                                                                                                                         |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Processeur | AMD Ryzen 9 8940HX with Radeon Graphics<br>Architecture : x86_64<br>Cœurs / Threads : 16 physiques / 32 logiques (2 threads/cœur)<br>Fréquence : 421.8 MHz (min) à 5386 MHz (max)<br>Cache L1d : 512 KiB total, répartis en 16 instances<br>Cache L1i : 512 KiB total, répartis en 16 instances<br>Cache L2 : 16 MiB total, répartis en 16 instances<br>Cache L3 : 64 MiB total, répartis en 2 instances |
| Mémoire    | 30 GiB de RAM système                                                                                                                                                                                                                                                                                                                                                                                    |
| Swap       | 8.0 GiB                                                                                                                                                                                                                                                                                                                                                                                                  |

Les expérimentations parallèles ont été réalisées sur cette même configuration matérielle.

#### 4.1.2 Environnement Logiciel / Software

Table 2: Configuration logicielle

| Composant              | Version / Détails                      |
|------------------------|----------------------------------------|
| Système d’exploitation | Fedora Linux 42 (Workstation Edition)  |
| Compilateur C/C++      | GCC 15.2.1 20250808 (Red Hat 15.2.1-1) |

Les optimisations de compilation ont été testées avec les flags `-O3 -march=native` pour activer l’auto-vectorisation et les optimisations spécifiques à l’architecture.

#### 4.1.3 Configuration Spécifique et Protocole

Afin de limiter la variabilité, nous appliquons un protocole strict de verrouillage de fréquence et d’affinité CPU. Les huit premiers cœurs physiques sont fixés à 4.0 GHz via `cpupower`, puis les exécutions sont épinglées sur ces cœurs avec `taskset`. Les commandes et scripts associés (`scripts/README.md`, `benchmark_affinity.sh`, `benchmark_large.sh`) sont fournis, ainsi qu’un exemple en Annexe B. Les temps sont mesurés par `tests/test_benchmark_large.cpp` à l’aide de `std::chrono`, avec un préchauffage (*warm-up*) systématique de 5 itérations.

**Maîtrise de l’aléatoire** Au-delà du contrôle matériel, les sources d’aléatoire sont verrouillées : l’initialisation des poids et le mélange des données utilisent une graine fixée via `-seed`. Cela garantit la reproductibilité des courbes et des métriques, et permet de comparer proprement initialisations et hyperparamètres.

## 4.2 Analyse des performances de calcul et d'entraînement

Cette section détaille les résultats obtenus lors des campagnes de test, en analysant l'impact du parallélisme et des optimisations bas niveau sur les performances. Il convient de préciser que ces micro-benchmarks portent exclusivement sur l'opération de multiplication matricielle (**matmul**), cœur intensif du calcul, et non sur le processus d'entraînement complet du réseau de neurones.

### 4.2.1 Passage à l'échelle (Thread Scaling)

L'évaluation du passage à l'échelle (*scaling*) de l'implémentation OpenMP a été réalisée en faisant varier le nombre de threads de 1 à 16, sur une architecture disposant de 8 cœurs physiques.

Afin de fiabiliser les mesures (en particulier pour les petites tailles), la boucle de *benchmark* est intégrée directement au programme C++ (`test_benchmark_large.cpp`), afin de minimiser les surcoûts de lancement. Les temps sont évalués avec un préchauffage (*warm-up*), un nombre d'itérations adaptatif et une agrégation robuste (moyenne tronquée).

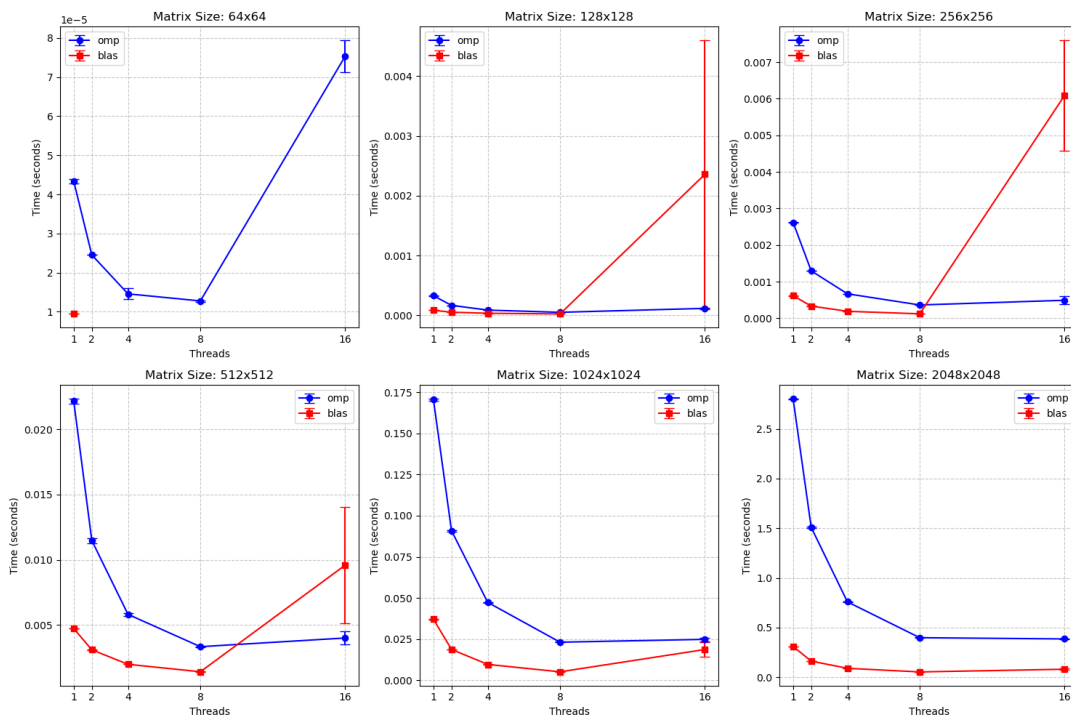


Figure 3: Temps d'exécution en fonction du nombre de threads pour différentes tailles de matrices.

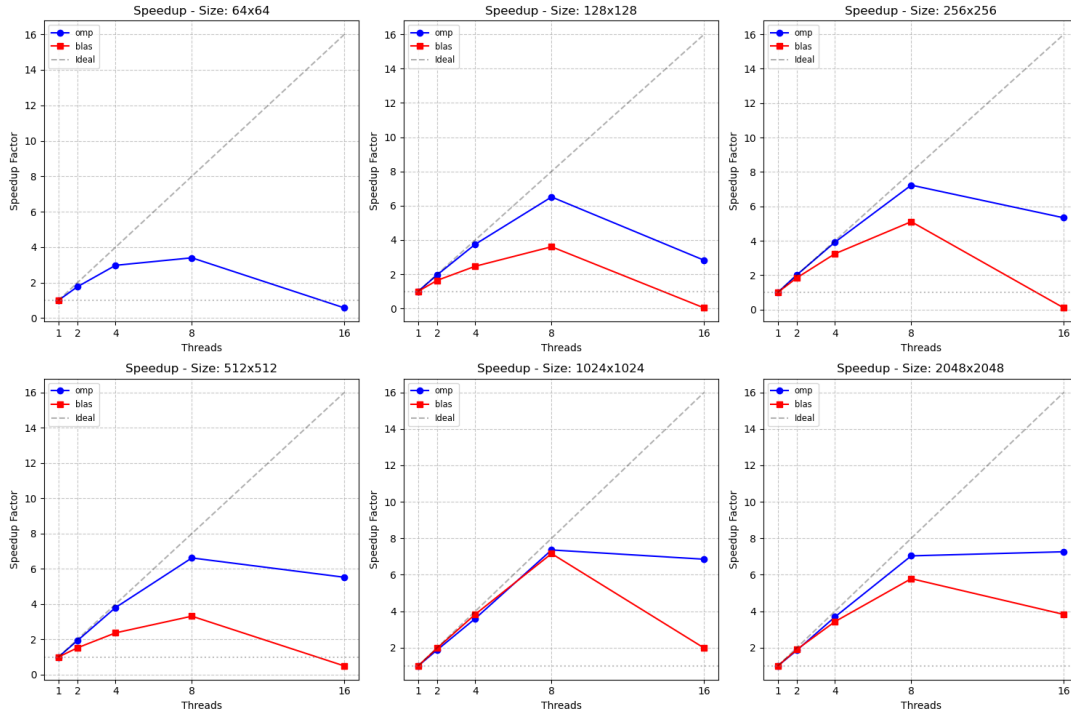


Figure 4: Speedup (accélération) en fonction du nombre de threads.

**Analyse des petites matrices ( $N = 64 \times 64$ ).** La parallélisation OpenMP apporte un gain conforme aux attentes entre 2 et 8 threads, mais devient contre-productive au-delà de 8 threads (au-delà du nombre de cœurs physiques). À cette échelle, la granularité du calcul est trop fine : la charge utile par thread devient comparable, voire inférieure, au surcoût de gestion (création, synchronisation et contention des ressources), ce qui annule les gains, voire dégrade les performances.

À l'inverse, BLAS reste monothread sur des matrices  $64 \times 64$ , la charge de calcul n'atteignant pas son seuil interne de déclenchement du parallélisme. Ainsi, pour cette échelle, les configurations les plus efficaces consistent soit à utiliser BLAS, soit à recourir à OpenMP en limitant le nombre de threads au nombre de cœurs physiques (8 threads dans notre cas).

**Matrices de taille intermédiaire ( $128 \times 128$  à  $1024 \times 1024$ ).** Pour les matrices de taille intermédiaire, OpenMP présente une accélération nette lorsque le nombre de threads augmente jusqu'au nombre de cœurs physiques disponibles. Au-delà de ce seuil, les gains se tassent, voire deviennent négatifs, sous l'effet de la saturation des ressources partagées et de l'augmentation des coûts de synchronisation.

BLAS offre de meilleures performances globales grâce à ses optimisations internes (vectorisation et découpage en blocs). Cependant, lorsque le nombre de threads dépasse le nombre de cœurs physiques, ses performances se dégradent et deviennent instables, principalement en raison de la contention des ressources et des changements de contexte (*context switching*) sur un même cœur.

**Grandes matrices ( $2048 \times 2048$ ).** Pour les matrices de grande taille, les deux approches bénéficient d'une parallélisation efficace jusqu'à environ 8 threads. Toutefois, l'augmentation du nombre de threads au-delà de ce point n'entraîne plus de gain significatif, le calcul devenant principalement limité par la bande passante mémoire. Dans ce contexte, BLAS reste la solution la plus performante et la plus stable.

Ainsi, l'efficacité du parallélisme dépend non seulement de la taille du problème, mais aussi d'une adéquation stricte entre nombre de threads et ressources matérielles disponibles.

#### 4.2.2 Hiérarchie des performances et Speedup

La comparaison globale des implémentations sur grandes matrices met en évidence cinq paliers de performance (Figure 5) :

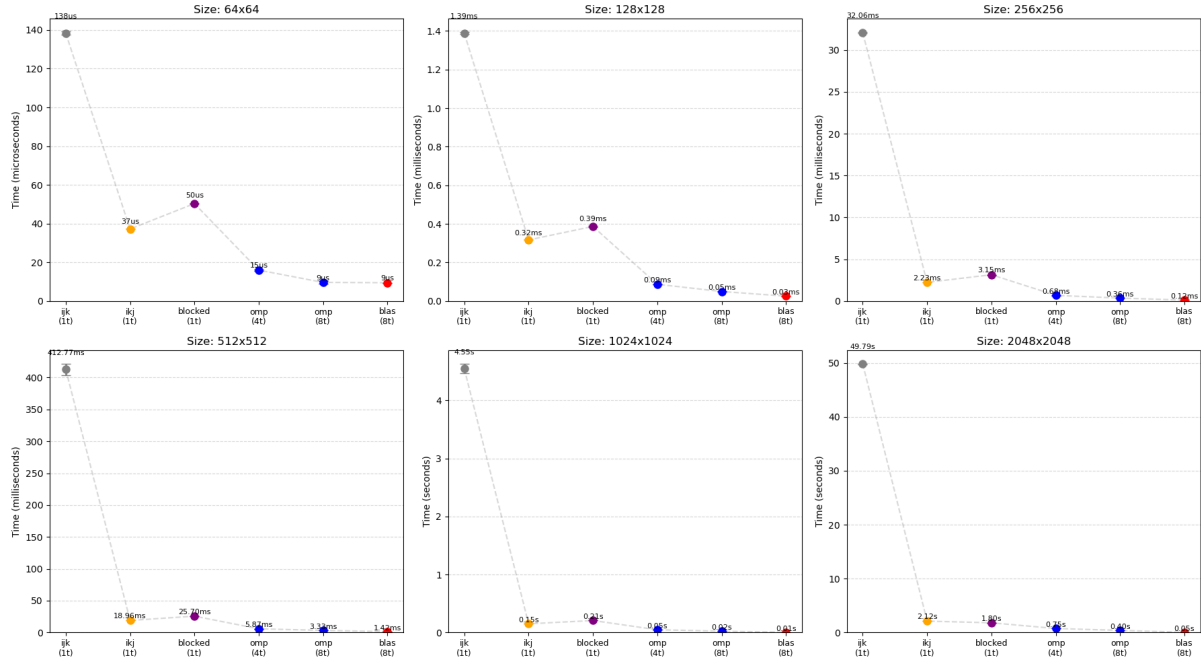


Figure 5: Comparaison des temps d'exécution (Naïf vs Reordered vs OpenMP vs BLAS).

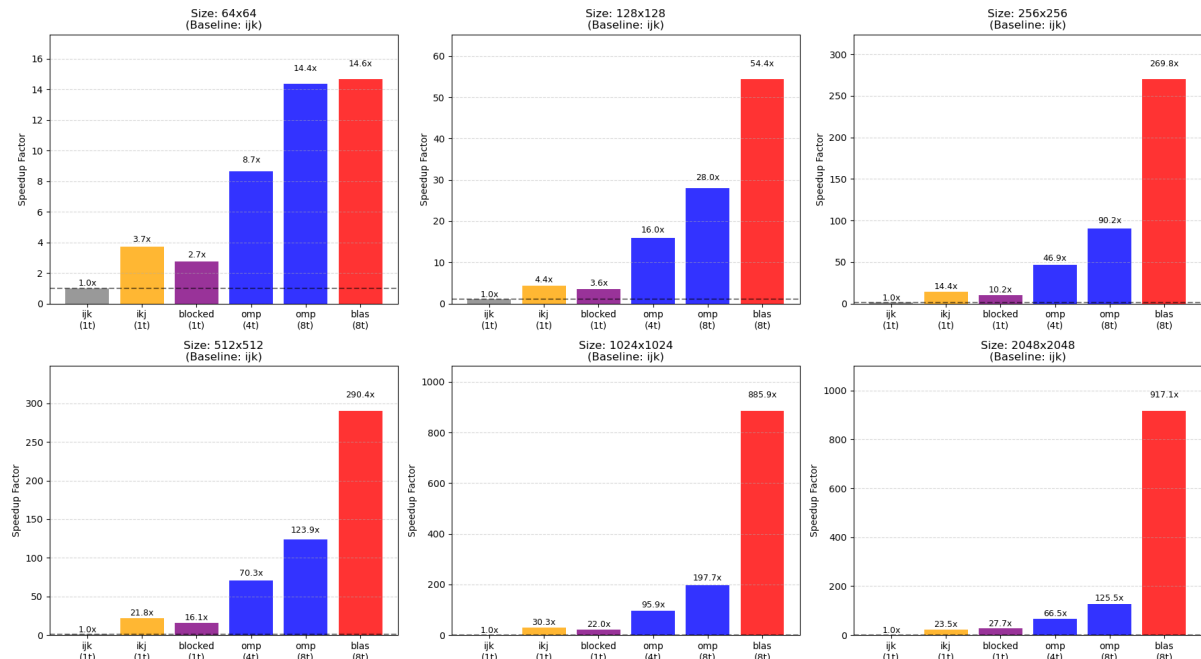


Figure 6: Speedup relatif des différentes optimisations.

1. **Naïf ( $ijk$ )** : L'ordre des boucles induit un parcours mémoire très défavorable, entraînant

de nombreuses fautes de cache. Le temps d'exécution devient prohibitif pour les grandes tailles, atteignant plusieurs secondes pour  $N = 2048$ .

2. **Réordonné (*ikj*)** : La permutation des boucles améliore la localité spatiale des accès mémoire et permet un premier gain significatif par rapport à l'implémentation naïve, en particulier pour les tailles intermédiaires et grandes.
3. **Bloqué (blocked)** : Cette optimisation apporte un gain mesurable par rapport à la version réordonnée, en améliorant la réutilisation des données dans les caches.
4. **OpenMP** : La parallélisation sur 8 cœurs permet d'exploiter le parallélisme matériel et apporte un gain supplémentaire, bien que celui-ci reste contraint par le nombre de cœurs physiques et par la hiérarchie mémoire.
5. **BLAS (OpenBLAS)** : Grâce à la combinaison d'optimisations avancées — blocage hiérarchique, vectorisation (AVX2/FMA) et micro-noyaux optimisés — BLAS surpasse largement les implémentations manuelles, avec un speedup pouvant atteindre plusieurs centaines de fois par rapport à la version naïve pour les plus grandes tailles.

#### 4.2.3 Analyse de Profiling et Loi d'Amdahl : vers l'entraînement complet

Afin de comprendre l'impact de ces optimisations sur le temps total d'apprentissage, nous avons profilé une époque complète d'entraînement (forward + backward) avec `gprof`.

**Architecture** : MLP  $784 \rightarrow 128 \rightarrow 10$  avec ReLU

**Taille du mini-batch** : 64

**Données** : MNIST Train (60 000 images)

**Avant optimisation (Kernel Naïf)** : L'application est totalement limitée par le calcul (*compute-bound*). La fonction `Matrix::matmul` consomme **environ 96%** du temps total (Tableau 3).

Table 3: Profil d'exécution (gprof) - Version Naïve : `Matrix::matmul` domine (96.4%)

| % Time | Self (s) | Calls | Function Name                             |
|--------|----------|-------|-------------------------------------------|
| 96.42  | 11.84    | 6256  | <code>Matrix::matmul</code>               |
| 1.06   | 0.13     | 1252  | <code>DataLoader::nextBatch</code>        |
| 0.73   | 0.09     | 3752  | <code>Matrix::transpose</code>            |
| 0.41   | 0.05     | 12216 | <code>Matrix::Matrix (constructor)</code> |
| 0.24   | 0.03     | 95M   | <code>Matrix::operator() const</code>     |
| 0.24   | 0.03     | 74M   | <code>Matrix::operator() (mut)</code>     |
| 0.24   | 0.03     | 9380  | <code>Node::addGrad</code>                |
| 0.24   | 0.03     | 1     | <code>MNISTDataset::load</code>           |
| 0.16   | 0.02     | -     | <code>_init</code>                        |
| 0.08   | 0.01     | 2504  | <code>MathOps::add</code>                 |

**Après optimisation (BLAS)** : Le temps passé dans `Matrix::matmul` s'effondre à moins de 1% ( $\approx 0.5s$ ). Le goulot d'étranglement se déplace alors vers :

**Chargement des données (`MNISTDataset::load`)**. Il représente  $\approx 43\%$  du temps. Ce coût élevé s'explique par l'allocation et l'initialisation à zéro (inhérente au constructeur `std::vector`) d'un volume important de mémoire (376 Mo), suivies d'une copie avec conversion de type (`uint8_t` vers `double`) pour chaque pixel.

**Allocations mémoire (`Matrix::Matrix`)**. Elles représentent  $\approx 29\%$  du temps.

Table 4: Profil d'exécution (gprof) - Version BLAS : `matmul` négligeable, overhead mémoire dominant

| % Time | Self (s) | Calls | Function Name                      |
|--------|----------|-------|------------------------------------|
| 42.86  | 0.03     | 1     | MNISTDataset::load                 |
| 28.57  | 0.02     | 12216 | Matrix::Matrix (init double)       |
| 28.57  | 0.02     | 938   | SGDOptimizer::step                 |
| 0.00   | 0.00     | 6256  | Matrix::matmul (optimisé par BLAS) |
| 0.00   | 0.00     | 95M   | Matrix::operator() const           |
| 0.00   | 0.00     | 74M   | Matrix::operator() (mut)           |
| 0.00   | 0.00     | 40k   | std::mersenne_twister              |
| 0.00   | 0.00     | 18k   | Matrix::Matrix (copy)              |
| 0.00   | 0.00     | 10k   | std::_Hashtable...                 |
| 0.00   | 0.00     | 10k   | Node::Node                         |

**Conséquence (Loi d'Amdahl) :** Bien que le noyau de calcul soit accéléré de plusieurs centaines de fois au niveau micro-benchmark, l'accélération *end-to-end* reste limitée par le chargement des données et les allocations mémoire. Dans nos mesures, une époque passe de 13.27s à 1.74s, soit un **speedup de 7.6x**. Ce résultat illustre la loi d'Amdahl [5] : après optimisation de `matmul`, les gains supplémentaires nécessitent d'agir sur la gestion mémoire et les entrées/sorties.

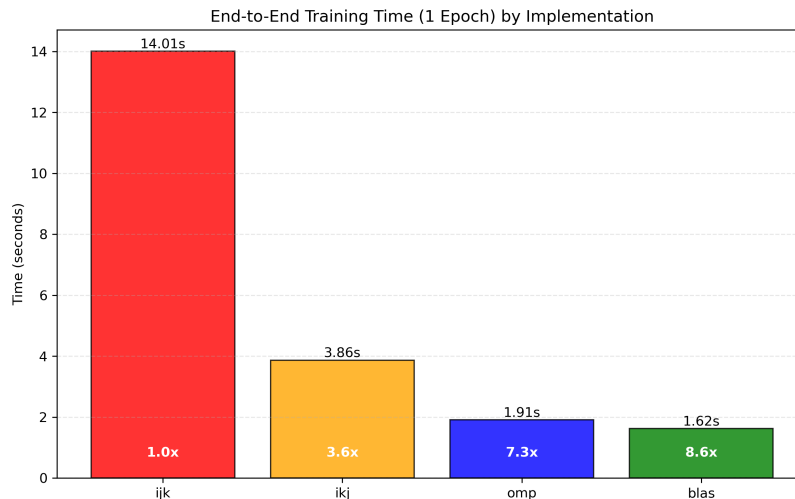


Figure 7: Répartition du temps d'exécution (End-to-End) : Naïf vs BLAS.

## 4.3 Convergence de l'entraînement

### 4.3.1 Premiers tests de convergence

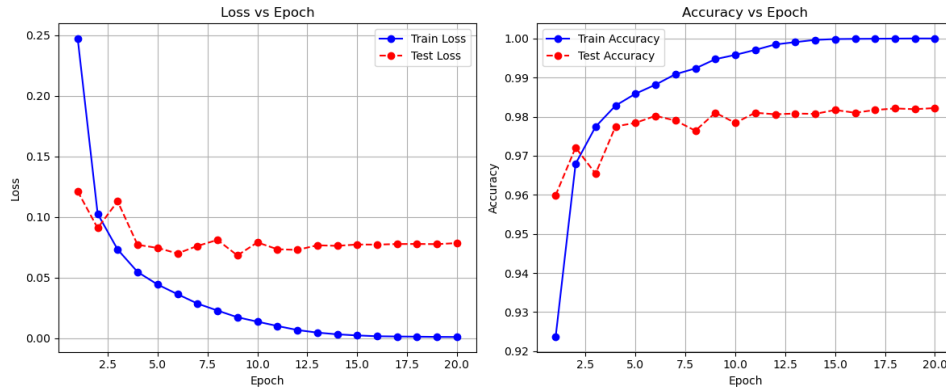
Afin de valider la correction du modèle et de l'algorithme d'optimisation, nous présentons la courbe d'apprentissage sur MNIST. L'ensemble des scripts et commandes utilisés pour reproduire ces expériences est disponible en Annexe C.

Les paramètres utilisés pour ce premier test sont :



| Paramètre                     | Valeur |
|-------------------------------|--------|
| Taux d'apprentissage ( $LR$ ) | 0.01   |
| Taille de batch ( $B$ )       | 64     |
| Taille cachée ( $H$ )         | 128    |
| Fonction d'activation         | ReLU   |
| Initialisation                | He     |
| Époques                       | 20     |

Table 5: Configuration standard pour le test de convergence.

Figure 8: Courbe de *loss* et de précision (*accuracy*) sur les données d'entraînement (60 000 images) et de validation (10 000 images).

**Commentaire sur la convergence (Figure 8) :** Cette figure présente deux graphiques : la *loss* (à gauche) et la précision (à droite).

**Courbe bleue (*train*).** Performance sur les données d'entraînement, reflétant la capacité du modèle à s'ajuster aux données vues.

**Courbe rouge (*validation*).** Performance sur les données de validation, jamais vues pendant l'entraînement. **Interprétation :** La courbe rouge suit la bleue mais reste légèrement en retrait (*loss* plus élevée, précision plus faible). C'est attendu : un modèle obtient généralement de meilleures performances sur les données d'entraînement que sur des données inédites. Si l'écart devenait très important, cela indiquerait un sur-apprentissage (*overfitting*) ; ici, il reste modéré, suggérant une généralisation correcte.

#### 4.3.2 Impact du Taux d'Apprentissage (Learning Rate)

Nous avons évalué l'impact du taux d'apprentissage sur la convergence du modèle en testant les valeurs  $\{0.025, 0.01, 0.0075, 0.005, 0.001\}$ . Les autres hyperparamètres ont été fixés : batch  $B = 64$ , couche cachée  $H = 128$ , activation ReLU et initialisation He. Pour assurer la robustesse statistique, chaque configuration a été exécutée trois fois avec des graines différentes.

La Figure 9 présente les résultats obtenus.

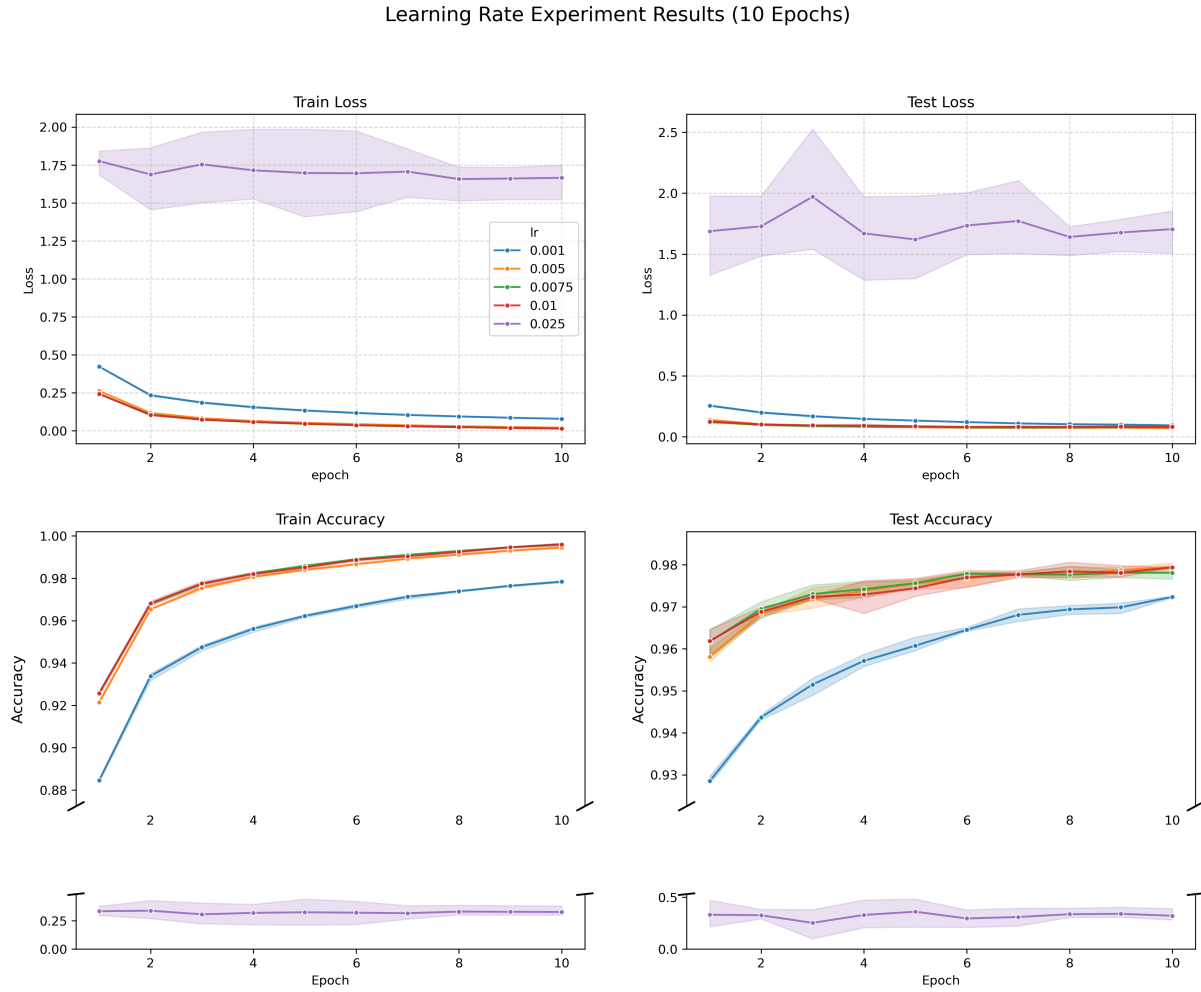


Figure 9: Impact du *learning rate* sur la convergence (10 époques, 3 graines). Les axes de précision utilisent une échelle discontinue pour visualiser simultanément les hautes précisions ( $> 90\%$ ) et les échecs de convergence ( $< 40\%$ ).

**Taux trop faible ( $LR = 0.001$ ).** Le modèle converge, mais trop lentement. Après 10 époques, la précision de validation plafonne à  $\sim 97.2\%$ , inférieure à celle obtenue avec des taux plus élevés, car les mises à jour des poids sont trop petites pour atteindre l’optimum rapidement.

**Zone optimale ( $LR \in [0.005, 0.01]$ ).** Ces taux offrent le meilleur compromis, avec une convergence rapide et stable, atteignant une précision de  $\sim 97.9\%$  à  $\sim 98.0\%$ . Nous retenons  $LR = 0.01$  pour sa rapidité.

**Taux trop élevé ( $LR = 0.025$ ).** On observe une instabilité critique (divergence) : les pas de gradient sont trop grands et l’optimiseur franchit les minima de la *loss*, conduisant à une précision d’environ  $\sim 32.1\%$  et à une variance élevée ( $\sigma \approx 0.06$ ), rendant l’entraînement inefficace.

Ces résultats expérimentaux confirment directement les intuitions théoriques présentées en section 2.3.2 concernant les risques de divergence (taux trop grand) et la lenteur de convergence (taux trop faible).

Nous retenons donc  $LR = 0.01$  comme valeur optimale pour la suite.

### 4.3.3 Impact de la Taille de Batch (Batch Size)

En fixant  $LR = 0.01$ , nous avons étudié l’impact de la taille de batch  $\{16, 32, 64, 128, 256\}$  sur la stabilité et la performance.

La Figure 10 illustre ces résultats.

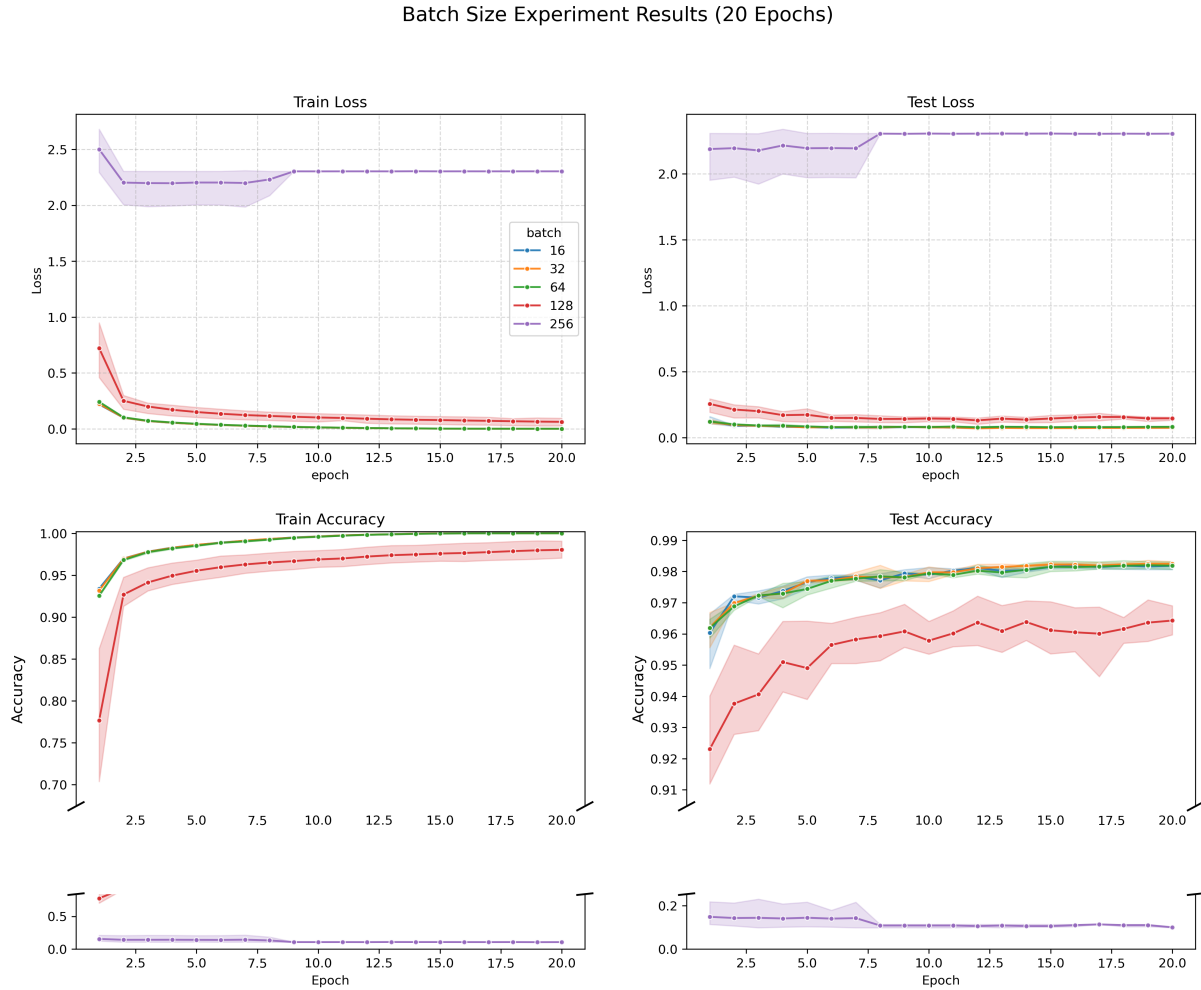


Figure 10: Impact de la Taille de Batch (20 époques,  $LR = 0.01$ , 3 graines). Les petits batchs (16, 32) convergent mieux mais sont plus lents à exécuter. Le batch 256 diverge avec ce  $LR$  fixe.

**Petits batchs (16, 32).** Ils offrent la meilleure précision finale ( $\sim 98.2\%$ ), vraisemblablement grâce au bruit introduit dans l'estimation du gradient, qui peut favoriser la généralisation. Aucune différence significative n'est observée entre 16 et 32.

**Batch moyen (64).** Il constitue un compromis, avec une précision comparable ( $\sim 98.2\%$ ) et une meilleure efficacité de calcul (parallélisation plus importante).

**Grands batchs (128, 256).** La performance se dégrade avec  $B = 128$ , et le modèle diverge avec  $B = 256$ . Pour stabiliser  $B = 256$ , il faudrait probablement ajuster le  $LR$  (*linear scaling rule*) ou utiliser un *warm-up*, ce qui n'est pas le cas ici avec  $LR = 0.01$  fixe.

*Note : Les grands batchs sont plus rapides à exécuter par époque, car les opérations matricielles plus larges exploitent mieux le parallélisme matériel (CPU/GPU) et la vectorisation. Les petits batchs génèrent davantage d'appels noyau (surcoût) et n'utilisent qu'une partie des unités de calcul, ce qui les rend plus lents malgré leur meilleure stabilité d'optimisation.*

#### 4.3.4 Impact de la Taille de la Couche Cachée (Hidden Size)

Enfin, en fixant  $LR = 0.01$  et un batch  $B = 64$ , nous avons évalué l'impact du nombre de neurones dans la couche cachée  $\{64, 128, 256\}$  sur la capacité du modèle.

La Figure 11 illustre ces résultats.

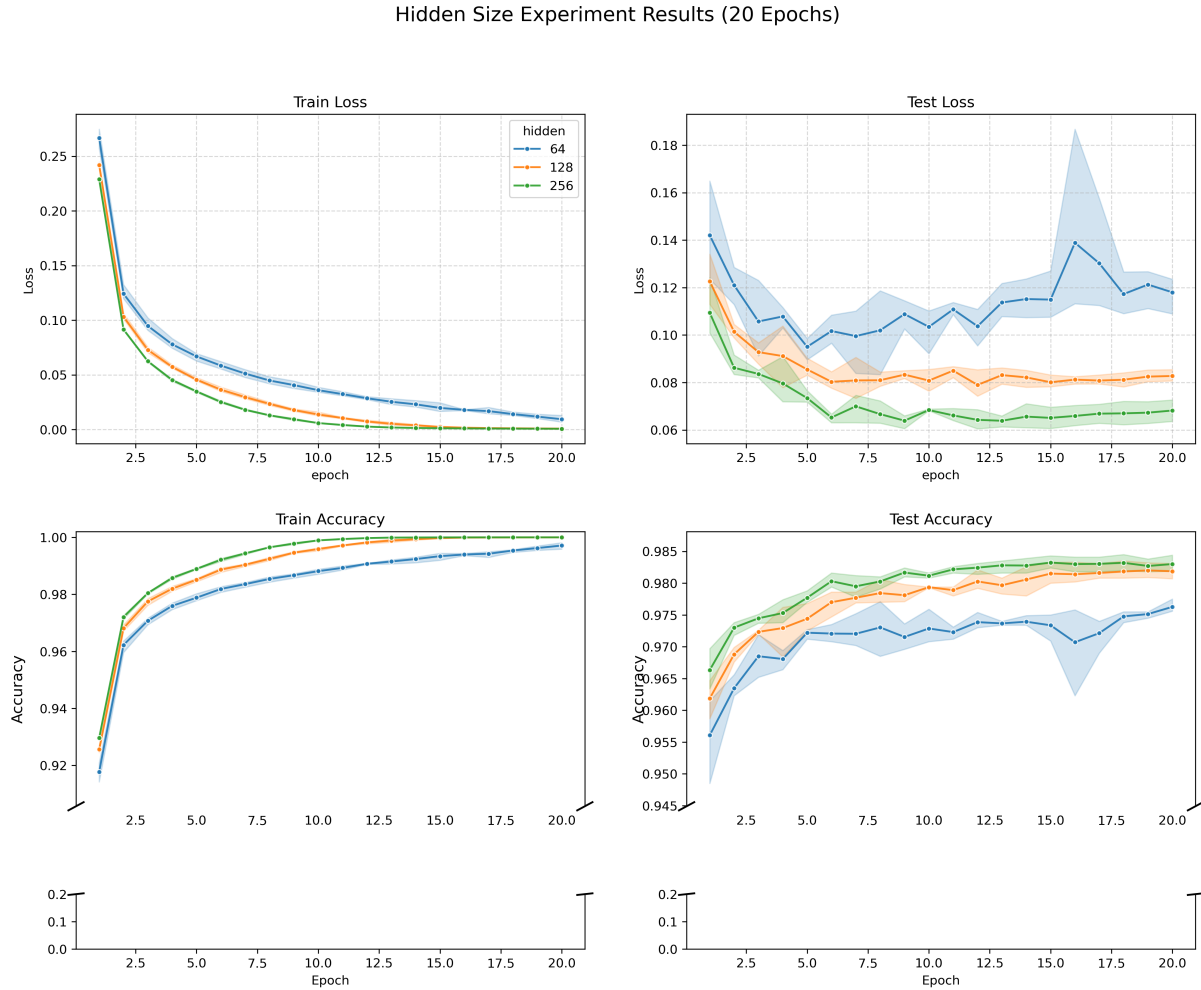


Figure 11: Impact de la Taille de la Couche Cachée (20 époques,  $LR = 0.01$ ,  $B = 64$ , 3 graines). L'augmentation de la taille améliore la précision, mais avec des rendements décroissants au-delà de 128 neurones.

**Capacité limitée ( $H = 64$ ).** Le modèle sous-apprend légèrement, avec une précision d'environ  $\sim 97.6\%$ . Cela suggère que 64 neurones ne suffisent pas à capturer toute la complexité de MNIST avec cette architecture.

**Compromis ( $H = 128$ ).** La précision atteint  $\sim 98.2\%$ , avec un gain significatif par rapport à  $H = 64$  sans accroître excessivement le coût de calcul.

**Rendements décroissants ( $H = 256$ ).** La meilleure précision est obtenue ( $\sim 98.3\%$ ), mais le gain ( $\approx 0.1\%$ ) reste marginal au regard du doublement du coût de calcul. Pour cette tâche,  $H = 128$  apparaît suffisant.

#### 4.3.5 Impact de la Fonction d'Activation et de l'Initialisation

Nous avons comparé les fonctions d'activation ReLU, Sigmoid et Tanh (avec  $LR = 0.01$ ,  $Batch = 64$ ,  $H = 128$ ). Pour garantir une comparaison équitable, nous avons utilisé l'initialisation de **He** pour ReLU et de **Xavier** pour Sigmoid/Tanh. De plus, nous avons inclus une initialisation dite "Manuelle" (*Manual Init*), correspondant à une distribution uniforme simple  $\mathcal{U} \sim [-0.1, 0.1]$ . Cette méthode naïve sert de point de référence (baseline) pour évaluer l'efficacité des méthodes modernes.

La Figure 12 présente les courbes de convergence.

Activation Function Experiment Results (20 Epochs)

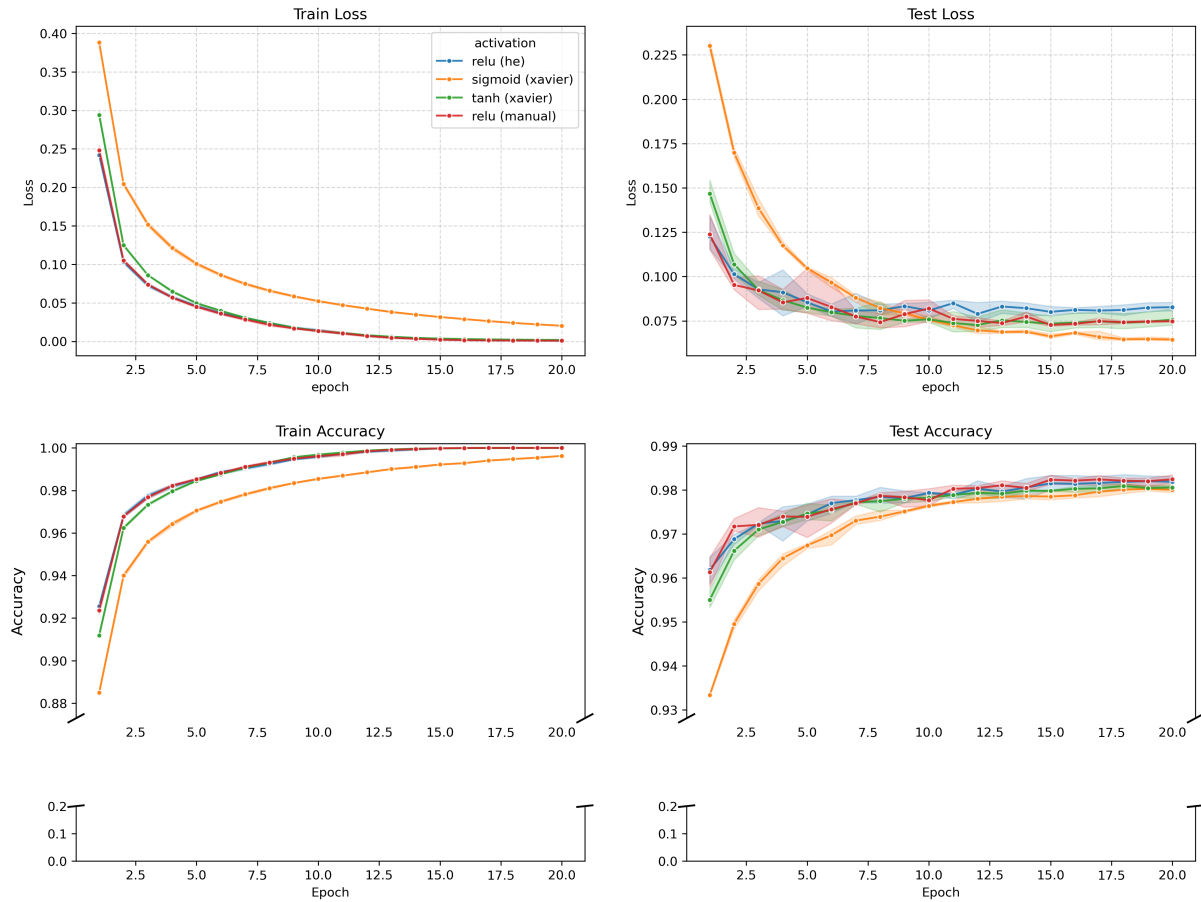


Figure 12: Comparaison des Fonctions d'Activation et Initialisation. ReLU (He) et ReLU (Manual) offrent les meilleures performances. Sigmoid et Tanh (avec Xavier) restent compétitifs grâce à une initialisation adaptée.

### Analyse des performances (précision de validation moyenne) :

**ReLU (Manual Init)** [ $\sim 98.24\%$ ]. Notre initialisation manuelle  $[-0.1, 0.1]$  obtient une performance comparable à l'initialisation de He. Cela s'explique par le fait que, pour  $N_{in} = 784$ , l'écart-type de cette distribution uniforme ( $\sim 0.057$ ) est proche de l'optimum théorique de He ( $\sim 0.05$ ).

**ReLU (He Init)** [ $\sim 98.19\%$ ]. La performance est quasi-identique à l'initialisation manuelle, confirmant la robustesse de ReLU sur ce problème.

**Tanh (Xavier)** [ $\sim 98.06\%$ ] et **Sigmoid (Xavier)** [ $\sim 98.00\%$ ]. Bien que légèrement inférieures à ReLU, ces activations restent compétitives ici car l'initialisation de Xavier limite la saturation des gradients au début de l'entraînement. Sans cette initialisation, la convergence aurait été sensiblement dégradée.

### Synthèse théorique :

**ReLU.** Standard actuel pour les **couches cachées** des réseaux profonds (CNN, MLP) : elle atténue le problème de disparition du gradient et reste très rapide à calculer.

**Sigmoid.** Principalement utilisée en **sortie** pour la classification binaire (probabilité  $p \in [0,1]$ ). Elle est généralement à éviter dans les couches cachées profondes en raison de sa saturation (gradient  $\rightarrow 0$ ).

**Tanh.** Souvent préférée à Sigmoid dans les couches cachées lorsque l'on souhaite des sorties centrées sur 0 (utile notamment pour certains RNN), mais elle souffre également de saturation.

**Conclusion finale des expériences :** La configuration optimale retenue est :  $LR = 0.01$ ,  $B = 64$ ,  $H = 128$ , **activation ReLU**, **init. He/Manual**.

#### 4.3.6 Impact de l’Affinité sur les charges d’entrée (Input Layer)

Les résultats présentés dans cette section analysent l’impact de l’affinité sur les opérations typiques de la couche d’entrée avec une taille de batch  $B = 64$  (soit une multiplication  $64 \times 784 \times 128$ , charge de travail comparable à une matrice carrée intermédiaire). Le tableau ci-dessous détaille les résultats pour 2 et 8 threads.

Table 6: Impact de l’affinité sur les performances pour une multiplication matricielle  $64 \times 784 \times 128$ .

| Threads | Affinité | Temps OMP ( $\mu s$ ) | Temps BLAS ( $\mu s$ ) | Ratio OMP/BLAS |
|---------|----------|-----------------------|------------------------|----------------|
| 2       | default  | 480                   | 132                    | 3.6x           |
| 2       | close    | 474                   | 128                    | 3.7x           |
| 2       | spread   | 487                   | 159                    | 3.1x           |
| 8       | default  | 149                   | 68                     | 2.2x           |
| 8       | close    | 152                   | 45                     | 3.4x           |
| 8       | spread   | 236                   | 75                     | 3.1x           |

L’analyse de ces données pour une multiplication  $64 \times 784 \times 128$  révèle :

**Impact de l’affinité sur BLAS.** Le mode `close` est déterminant : à 8 threads, il réduit le temps à  $45\mu s$ , contre  $68\mu s$  en configuration `default`, soit un gain d’environ **33%**. Le mode `spread` est le moins performant ( $75\mu s$ ), car il disperse les threads et pénalise la localité des caches partagés.

**Impact sur OpenMP.** Notre implémentation réagit différemment : à 8 threads, `default` ( $149\mu s$ ) et `close` ( $152\mu s$ ) sont comparables, tandis que `spread` dégrade fortement les performances ( $236\mu s$ ). Cela suggère que le placement par défaut est déjà relativement compact, et que forcer la dispersion nuit à la cohérence de cache de notre algorithme.

**Conclusion :** Pour les opérations de la couche d’entrée ( $64 \times 784 \times 128$ ), l’utilisation de bibliothèques optimisées comme BLAS est impérative. Cependant, ces bibliothèques sont très sensibles à la localité : pour obtenir le gain de performance maximal (facteur  $\times 4$  face à OpenMP), il est crucial de configurer l’affinité des threads (mode `close`) afin de minimiser les latences cache.

#### 4.4 Synthèse des Résultats Expérimentaux

L’ensemble de ces expérimentations met en lumière la dualité du défi posé par les réseaux de neurones : un défi **algorithmique** (trouver les bons hyperparamètres pour la convergence) et un défi **computationnel** (exécuter ces calculs efficacement).

**Aspect algorithmique :** La recherche par grille a permis d’isoler une configuration robuste (**LR=0.01, Batch=64, H=128, ReLU**) capable d’atteindre  $\sim 98.2\%$  de précision sur MNIST, validant notre implémentation de la rétropropagation. **Aspect performance :** L’optimisation bas niveau s’avère critique. L’utilisation de bibliothèques vectorisées (BLAS) avec une affinité CPU contrôlée (`close`) offre des gains de performance considérables sur le noyau de calcul ; dans nos mesures, la configuration la plus efficace correspond à **8 threads** avec `OMP_PROC_BIND=close`. Cependant, l’accélération globale de l’apprentissage reste bornée par les coûts de gestion mémoire et de chargement de données (loi d’Amdahl), soulignant l’importance d’une optimisation holistique du système.

## 5 D  roul   du Projet

L'  tat de l'art a   t   r  dig   conjointement par Jianye, Hao et Xiang, afin de pr  senter le contexte th  orique du projet et les approches existantes en mati  re de r  seaux de neurones et de classification sur MNIST. L'  quipe a ensuite organis   son travail en plusieurs phases successives, chacune port  e par un ou plusieurs membres selon leurs affinit  s techniques.

La phase de conception a   t   initi  e par Jianye, qui a r  dig   la documentation pr  liminaire et produit la premi  re version du diagramme UML, permettant de clarifier l'architecture g  n  rale du projet. La mise en place du premier prototype fonctionnel du MLP a ensuite   t   assur  e par Abdennour, qui a d  velopp   les modules fondamentaux li  s aux tenseurs, aux couches lin  aires et aux fonctions d'activation.

La r  alisation du moteur de diff  renciation automatique a   t   r  partie entre Hao et Xiang, chacun prenant en charge une partie des op  rations n  cessaires au calcul du graphe et des gradients. Par la suite, Jianye a int  gr   l'ensemble de ces composants, g  n  r   une seconde version du diagramme UML pour refl  ter les ajustements apport  s, et affin   la conception en fonction des probl  mes rencontr  s.

Le d  veloppement du m  canisme d'entra  nement a   t   amorc   par Xiang, tandis que Jianye et Hao ont poursuivi respectivement l'impl  mentation de l'optimiseur et des fonctions de *loss*. L'int  gration du jeu de donn  es MNIST et la validation du pipeline complet ont   t   assur  es par Jianye.

Jianye a   galement men   des tests de performance et des analyses de profilage afin d'optimiser l'ensemble du syst  me, ainsi qu'une recherche exp  rimentale visant    identifier une configuration robuste des hyperparam  tres.

Enfin, la r  daction du rapport a   t   r  alis  e collectivement.

## 6 Conclusion & perspectives

### 6.1 Conclusion :

Ce projet a permis de d  mystifier le fonctionnement interne des r  seaux de neurones profonds en impl  mentant,    partir de z  ro (*from scratch*), un moteur complet d'apprentissage en C++. L'architecture retenue, bas  e sur un graphe de calcul dynamique (DAG) et la diff  renciation automatique, a d  montr   la flexibilit   du C++ moderne (pointeurs intelligents, polymorphisme) pour mod  liser des concepts math  matiques abstraits tout en garantissant une gestion rigoureuse de la m  moire.

Sur le plan de la performance, l'analyse exp  rimentale a mis en   vidence l'impact critique des optimisations bas niveau. Si le noyau de calcul (`matmul`) a b  n  fici   d'une acc  l  ration spectaculaire de plus de trois ordres de grandeur gr  ce    BLAS et OpenMP, l'acc  l  ration globale de l'application est rest  e plus modeste ( $7.6\times$ ). Ce contraste illustre la loi d'Amdahl : une fois le calcul matriciel optimis  , le goulot d'  tranglement se d  place in  vitablement vers la gestion des donn  es (*I/O*) et les allocations m  moire.

**Limites et menaces    la validit   :** Les conclusions de performance d  pendent d'un protocole de mesure contr  l   (verrouillage de fr  quence, affinit   CPU) et peuvent varier sur d'autres architectures, avec d'autres impl  mentations BLAS, ou sous une charge syst  me diff  rente. Par ailleurs, l'analyse se concentre principalement sur le noyau `matmul` dense et ne couvre pas n  cessairement l'ensemble des co  ts d'un entra  nement complet (pipeline de donn  es, allocations, et surco  ts d'orchestration). Enfin, le choix d'un MLP sur MNIST limite l'extrapolation    des architectures plus modernes (CNN) et    des jeux de donn  es plus complexes.

### 6.2 Perspectives :

Plusieurs axes d'am  lioration pourraient enrichir ce projet :

**Support GPU (CUDA).** Déporter les calculs matriciels sur GPU afin d’accélérer fortement l’entraînement, tout en minimisant les transferts hôte–périphérique.

**Vectorisation explicite (SIMD).** Remplacer l’auto-vectorisation du compilateur par des intrinsèques (AVX-512) ou des micro-noyaux optimisés pour exploiter pleinement les unités de calcul.

**Optimisation mémoire.** Mettre en place un *memory pool* ou une arène d’allocation pour réduire le surcoût de `malloc/free` mis en évidence par le profilage.

**Extensions à l’échelle.** Explorer le **parallélisme de modèles** (exécution de plusieurs instances pour l’exploration d’hyperparamètres), l’**extension aux CNN** pour traiter des jeux de données plus complexes, ainsi que le **parallélisme distribué (MPI)** pour paralléliser le traitement des *mini-batches* (*data parallelism*).



## 7 Références

### References

- [1] Ian Goodfellow, Yoshua Bengio, et Aaron Courville. *Deep Learning*. MIT Press, 2016. 6
- [2] Yann LeCun, Léon Bottou, Yoshua Bengio, et Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. 4, 7
- [3] <https://m1-chps.github.io/glhpc/UVSQ,lecture5>, 2025. Consulté le 17/12/2025. 3, 5
- [4] David E. Rumelhart, Geoffrey E. Hinton, et Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986. 6
- [5] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Conference Proceedings*, 30:483–485, 1967. 16
- [6] OpenMP Architecture Review Board. *OpenMP Application Program Interface Version 5.0*. 2018.
- [7] Jack J. Dongarra et al. A set of level 3 basic linear algebra subprograms. *ACM Transactions on Mathematical Software (TOMS)*, 16(1):1–17, 1990.

## 8 Glossaire

### Contexte et État de l’art

|                            |                                                                                               |
|----------------------------|-----------------------------------------------------------------------------------------------|
| <b>Autodifférentiation</b> | Calcul automatique des gradients en appliquant la règle de la chaîne sur un graphe de calcul. |
| <b>MLP</b>                 | <i>Multi-Layer Perceptron</i> : réseau <i>feed-forward</i> (couches linéaires + activations). |
| <b>MNIST</b>               | Jeu de données de chiffres manuscrits (28×28, 10 classes) utilisé pour la classification.     |
| <b>HPC</b>                 | <i>High-Performance Computing</i> : techniques visant à maximiser le débit de calcul.         |
| <b>Backpropagation</b>     | Propagation inverse des gradients depuis la <i>loss</i> vers les paramètres.                  |

### Architecture et Modèles (CNN)

|            |                                                                                                                           |
|------------|---------------------------------------------------------------------------------------------------------------------------|
| <b>CNN</b> | <i>Convolutional Neural Network</i> : réseau utilisant des couches de convolution, adapté aux données spatiales (images). |
|------------|---------------------------------------------------------------------------------------------------------------------------|

### Implémentation : Tenseurs et Réseaux

|                     |                                                                                                                        |
|---------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>Tensor/Matrx</b> | Structure de données pour stocker des valeurs et effectuer des opérations numériques (ici, matrices denses contiguës). |
| <b>RNN</b>          | <i>Recurrent Neural Network</i> : réseau récurrent, adapté aux données séquentielles.                                  |

**Implémentation : Optimisation**

**SGD** *Stochastic Gradient Descent* : optimisation par mise à jour des paramètres à partir du gradient estimé sur un *mini-batch*.

**Implémentation : Calcul et Matériel**

**CPU** *Central Processing Unit* : processeur généraliste exécutant le programme.

**SIMD** *Single Instruction, Multiple Data* : exécution vectorielle de la même opération sur plusieurs données.

**BLAS** *Basic Linear Algebra Subprograms* : interface standard pour l'algèbre linéaire (p. ex. produits matriciels).

**OpenMP** API de parallélisme à mémoire partagée pour C/C++ et Fortran.

**DAG** *Directed Acyclic Graph* : graphe orienté sans cycle, utilisé comme graphe de calcul.

**Logits** Scores réels produits par le modèle avant normalisation (p. ex. avant *softmax*).

**One-hot** Encodage d'une classe par un vecteur binaire de dimension  $K$  (un seul 1, le reste à 0).

**Softmax** Transformation d'un vecteur de scores en probabilités (classification multi-classes).

**Epoch** Passage complet sur le jeu d'entraînement.

**Expérimentations : Configuration**

**Affinité CPU** Restriction du placement des threads/processus à un sous-ensemble de cœurs pour réduire la variabilité.

**Warm-up** Itérations de préchauffage avant mesure (stabilisation caches et exécution).

**Expérimentations : Résultats**

**Speedup** Accélération : rapport entre un temps de référence et un temps optimisé (p. ex.  $T_1/T_p$ ).

**I/O** *Input/Output* : lecture/écriture de données, pouvant devenir un goulot d'étranglement.

**Perspectives (GPU)**

**CUDA** Plateforme et modèle de programmation NVIDIA pour l'exécution sur GPU.

**GPU** *Graphics Processing Unit* : processeur massivement parallèle, efficace pour l'algèbre linéaire.

**Autres concepts**

**Overfitting** Sur-apprentissage : le modèle s'ajuste trop aux données d'entraînement et généralise moins bien.

## A Exemple : graphe de calcul et autodiff

Un **graphe de calcul** représente une expression comme un enchaînement d'opérations élémentaires. Par exemple, pour

$$z = x \cdot w, \quad y = z + b, \quad L = y^2,$$

le graphe contient trois nœuds d'opération (multiplication, addition, carré) reliés par des dépendances ( $x, w \rightarrow z \rightarrow y \rightarrow L$ ). Lors du **passage avant**, chaque nœud calcule puis *stocke* sa valeur (ici  $z$  et  $y$ ), afin d'éviter de recalculer ces intermédiaires lors du backward. Lors du **passage arrière** (autodiff en mode inverse), chaque nœud utilise sa **dérivée locale** et le gradient amont pour mettre à jour ses parents :

$$\frac{\partial L}{\partial y} = 2y, \quad \frac{\partial y}{\partial z} = 1, \quad \frac{\partial y}{\partial b} = 1, \quad \frac{\partial z}{\partial x} = w, \quad \frac{\partial z}{\partial w} = x.$$

On obtient alors, par composition,  $\frac{\partial L}{\partial b} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z}$ ,  $\frac{\partial L}{\partial z} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z}$ ,  $\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \cdot w$  et  $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z} \cdot x$ . Dans notre implémentation, cette logique se traduit par des nœuds **Node** stockant une **value** (résultat du forward) et une fonction **backwardFn\_** (dérivée locale + accumulation du gradient dans les parents). Enfin, la **rétropropagation** correspond au cas particulier où ce graphe de calcul est celui d'un réseau de neurones : c'est l'application de l'autodiff en mode inverse à une composition de couches.

## B Protocole de Verrouillage des Fréquences

Afin de garantir la reproductibilité des mesures, le script suivant est exécuté avant chaque campagne de test pour fixer la fréquence du CPU à 4.0 GHz sur les cœurs physiques.

Listing 3: Commandes de configuration CPU (mode performance)

```
# 1. Passer le gouverneur en mode performance
sudo cpupower frequency-set -g performance

# 2. Verrouiller la fréquence min et max à 4.0 GHz
# (Adaptez la valeur selon votre CPU)
sudo cpupower frequency-set -u 4000MHz -d 4000MHz

# 3. Vérifier la fréquence
cpupower frequency-info | grep "current_CPU_frequency"
```

Une fois les mesures terminées, l'environnement est restauré :

Listing 4: Restauration de l'environnement

```
sudo cpupower frequency-set -d 421MHz -u 5386MHz
sudo cpupower frequency-set -g powersave
```

## C Scripts d'Expérimentation

L'ensemble des expériences (Grid Search) a été automatisé via des scripts Bash invoquant l'exécutable **ppn\_train** avec les hyperparamètres appropriés. Voici un exemple représentatif pour l'étude de la taille de couche cachée :

Listing 5: Exemple de script d'automatisation (exp\_hidden\_size.sh)

```
#!/bin/bash
# Configuration
EXE="./build/ppn_train"
EXP_DIR="output/experiments/hidden"
```

```
mkdir -p "$EXP_DIR"

# Hyperparameters to test
# Taille de couche cachée à tester
# On peut aussi varier le taux d'apprentissage, le batch size, activation
  function et initialisation.
HIDDEN_SIZES=(64 128 256)
SEEDS=(42 43 44)
EPOCHS=20

# Fixed parameters (Control Variates)
LR=0.01
BATCH=64
ACTIVATION="relu"
INIT="he"

for hidden in "${HIDDEN_SIZES[@]}; do
  for seed in "${SEEDS[@]}; do
    # Run command
    $EXE --learning_rate $LR \
        --batch_size $BATCH \
        --hidden_size $hidden \
        --activation $ACTIVATION \
        --init $INIT \
        --epochs $EPOCHS \
        --seed $seed \
    > "$EXP_DIR/hidden_${hidden}_seed${seed}.log"
  done
done
```